

Senior Full-Stack Developer Technical Assessment

Project: Mini Doctor Appointment App + RAG Chatbot

Estimated effort: 16–20 hours

AI-assisted development is explicitly allowed and expected.

Candidates may use tools such as Cursor, Claude Code, GitHub Copilot, ChatGPT, or other AI coding assistants.

The purpose of this assessment is **not** to evaluate how quickly a candidate can manually write code. We want to evaluate whether a senior developer can use modern development tools effectively while demonstrating strong understanding of:

- Full-stack architecture
- Backend/API design
- Payload CMS
- MongoDB/data modeling
- Ionic React and Android
- Authentication and authorization
- Concurrency and data integrity
- RAG architecture
- LLM integration
- AI reliability and hallucination prevention
- Error handling
- Maintainable code
- Technical decision-making

The candidate must be able to explain the implementation and the decisions made, including where AI tools were used.

1. Objective

Build a small healthcare application consisting of:

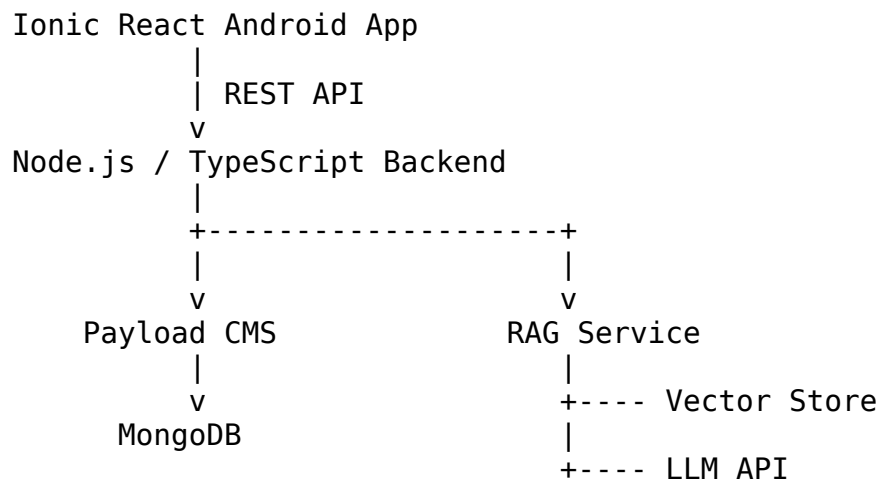
1. An **Ionic React Android application**

2. A **Node.js/TypeScript backend using Payload CMS**
3. A **MongoDB database**
4. A **RAG-based chatbot using the provided healthcare documents**
5. An **LLM integration**
6. An **admin workflow for unanswered chatbot questions**

The UI does not need to be visually sophisticated. We are primarily evaluating the technical implementation, architecture, correctness, and reasoning.

2. Required Architecture

The expected high-level architecture is:



Required technologies

The implementation must use:

- **Ionic React**
- **Android**
- **Node.js / TypeScript**
- **Payload CMS**
- **MongoDB**
- **REST API**
- **LLM API**
- **Vector database/vector store**

The candidate may choose the specific RAG framework, embedding model, vector database, and LLM provider.

The candidate must document why those technologies were selected.

3. Feature A — Doctor Appointment System

3.1 Doctor Management

Doctors must be managed through Payload CMS.

Each doctor should have:

- Name
- Specialization
- Qualifications
- Profile photo
- Short bio
- Active/inactive status

Seed the system with at least **2-3 doctors**.

Administrators should be able to manage doctors through the Payload admin panel.

3.2 Appointment Slots

Each appointment slot should contain:

- Doctor
- Date
- Time
- Status

The candidate may either:

- Pre-create appointment slots, or
- Implement a simple slot-generation mechanism.

A sophisticated scheduling engine is not required.

3.3 Appointment Booking

A logged-in patient must be able to:

1. View doctors
2. View doctor details
3. View available slots
4. Select a slot
5. Book an appointment
6. View the booked appointment

Appointment data should contain:

- Patient
- Doctor
- Slot
- Status
- Created timestamp

Possible appointment statuses:

- Booked
- Cancelled

Cancellation is optional.

4. Authentication

The application must support:

Registration

The user provides:

- Name
- Email
- Password

Login

The user logs into the application.

Payload authentication may be used.

No sophisticated user-management system is required.

5. Required Mobile Screens

The Ionic application should contain at least:

Login

- Email
- Password
- Login
- Link to registration

Registration

- Name
- Email
- Password

Home

A simple navigation/dashboard screen.

Doctors

Display:

- Doctor photo
- Name
- Specialization

Doctor Details

Display:

- Name
- Specialization

- Qualifications
- Bio
- Available slots

Book Appointment

Allow the user to select an available slot and confirm the booking.

Booking Confirmation

Clearly show that the appointment was successfully booked.

My Appointment

Display:

- Doctor
- Date
- Time
- Appointment status

Chatbot

A conversational interface allowing the user to ask healthcare-related questions.

A sophisticated calendar UI is **not required**.

6. Important Requirement — Prevent Double Booking

The system must prevent two users from successfully booking the same appointment slot.

For example:

User A ---> Book Doctor X / 10:00
User B ---> Book Doctor X / 10:00

Only one request may succeed.

The other request must receive an appropriate error such as:

This appointment slot is no longer available.

The candidate must implement this protection on the **backend/database side**.

A client-side check alone is not sufficient.

The candidate must explain in the README:

- How double booking is prevented
- Where the validation happens
- How concurrent requests are handled
- Any database constraints/transactions/atomic operations used

This is an important part of the assessment.

7. Feature B — RAG Healthcare Chatbot

The chatbot is a **mandatory part of the assignment**.

The candidate will receive **2-3 small PDF documents** containing healthcare information.

Example documents:

- Pregnancy Nutrition Guide
- Prenatal Care Guide
- Child Nutrition Guide

The chatbot must answer questions based on these documents.

8. RAG Document Ingestion

The candidate must implement an ingestion process similar to:

```
PDF
|
v
Extract Text
|
v
Chunk Text
|
v
```

Create Embeddings



Store in Vector Store

The ingestion process may be implemented as:

- A script
- An API endpoint
- A command
- A simple application workflow

A sophisticated document-management platform is not required.

The candidate should document:

- PDF extraction approach
 - Chunking strategy
 - Embedding model
 - Vector store
 - Metadata stored with chunks
 - How documents are loaded/re-indexed
-

9. RAG Query Flow

When a user asks a question, the system should perform approximately:

User Question



Create Query Embedding



Vector Search



Relevant Chunks



LLM



Grounded Answer

The LLM must be instructed to answer using the retrieved context.

The system should avoid generating information that is not supported by the retrieved documents.

Where practical, the chatbot should identify the source document used for the answer.

For example:

Answer:

...

Source:

Prenatal Care Guide.pdf

Exact formatting is not important.

10. Critical Requirement — Unknown Questions

The chatbot must **not hallucinate an answer** when the required information is not available in the knowledge base.

For example, if the documents contain information about nutrition but the user asks for a medication dosage that is not covered by the documents, the chatbot should not invent a dosage.

Instead, it should respond with an appropriate fallback, such as:

I don't have enough information in the available healthcare documents to answer that question. The question has been submitted for review.

The exact wording is up to the candidate.

The important behavior is:

If the system does not have sufficient knowledge, it must say so rather than fabricate an answer.

11. Unresolved Query Storage

When the chatbot cannot confidently answer a question, the question must be stored in Payload CMS.

Create an **Unresolved Queries** collection containing:

- Question
- User
- Status
- Created timestamp
- Human response

Status:

- New
- Resolved

Example:

Question:

"What medication dosage should I take?"

Status:

New

Created:

2026-XX-XX

Human Response:

(empty)

12. Human Resolution Workflow

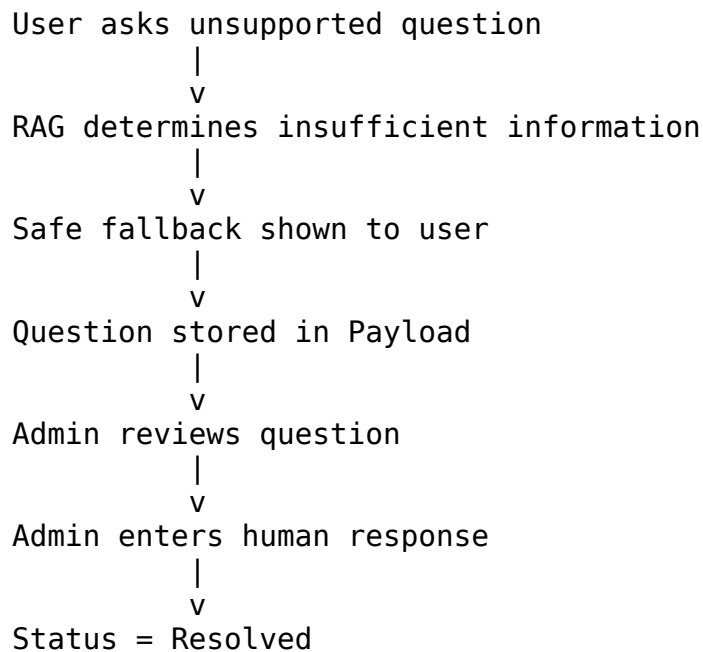
An administrator must be able to view unresolved chatbot questions through Payload CMS.

The administrator should be able to:

1. View the question
2. Enter a human response
3. Change the status to Resolved

No notification system is required.

The workflow should be:



13. RAG Relevance / Fallback Decision

The candidate must implement a reasonable mechanism for determining whether the retrieved information is sufficient to answer the question.

Possible approaches include:

- Vector similarity threshold
- Similarity score
- LLM relevance check
- Combination of retrieval score + LLM evaluation
- Another reasonable approach

There is no single required library or algorithm.

However, the candidate must explain:

How does your system decide whether it knows enough to answer the question?

This is an important part of the evaluation.

We are looking for evidence that the candidate understands that:

Vector search returning something

≠

The system necessarily has enough information to answer.

14. Chatbot Error Handling

The chatbot should handle at least:

No relevant information

Show the safe fallback and store the unresolved query.

LLM/API failure

Show a reasonable user-facing error.

Network failure

Show a reasonable user-facing error.

The application should not expose internal API keys, stack traces, or sensitive implementation details to users.

15. Security Requirements

At minimum:

- LLM API keys must remain on the backend
- No secrets committed to Git
- Payload/admin access must be protected
- Authentication must be implemented
- Backend inputs should be validated
- A user must not be able to access another user's appointments
- API authorization should be enforced server-side

Advanced penetration testing is not required.

16. Error Handling

The application should handle common failures gracefully.

Appointment

- Slot already booked
- Invalid booking
- Unauthorized booking
- Network failure

Chatbot

- No relevant knowledge
- LLM/API failure
- Network failure

The UI should provide understandable error messages rather than simply failing silently.

17. Required Deliverables

The candidate must submit:

```
/project
  /mobile
  /backend
  /rag
  README.md
```

The exact project structure may differ if the candidate has a reasonable alternative.

Android Application

Provide either:

- Android debug APK

or

- Clear instructions for building the APK

Payload CMS

The Payload admin panel must be runnable and demonstrate:

- Doctors
- Appointment slots
- Appointments
- Unresolved queries

README

The README must contain:

1. Setup instructions
2. Environment variables
3. MongoDB setup
4. Payload setup
5. Ionic setup
6. How to run the backend
7. How to run the mobile application
8. How to load/index the RAG documents
9. Embedding approach
10. Vector store
11. LLM configuration
12. Architecture diagram
13. Technical decisions

The candidate must specifically explain:

- RAG architecture
 - Chunking strategy
 - Retrieval strategy
 - How the system decides when it has insufficient information
 - How hallucinations are reduced
 - How double booking is prevented
 - Authentication/authorization approach
 - How unanswered questions are handled
-

18. AI-Assisted / Vibe Coding Requirement

AI coding tools are **allowed and expected**.

The candidate may use:

- Cursor
- Claude Code
- GitHub Copilot
- ChatGPT
- Gemini
- Other AI development tools

We are not evaluating whether the candidate manually typed every line.

We are evaluating whether the candidate can:

- Define the architecture
- Break the problem into appropriate tasks
- Give effective instructions to AI tools
- Review generated code
- Detect incorrect implementations
- Debug AI-generated code
- Make architectural decisions
- Validate security and correctness
- Understand the final implementation
- Modify the implementation when requirements change

The candidate must disclose in the README:

AI Usage

Briefly describe:

- Which AI tools were used
- What parts were generated/assisted by AI
- What parts were implemented manually
- How AI-generated code was reviewed and validated
- Any significant AI-generated mistakes that were identified and corrected

The candidate should be able to explain the important parts of the system during the technical review.

19. Evaluation Criteria

Area	Weight
Architecture & Backend/API Design	20%
Payload CMS & Data Modeling	15%
Appointment Booking & Concurrency	15%
RAG & LLM Implementation	20%
Ionic React / Android	10%
Authentication, Security & Error Handling	10%
Code Quality, README & Technical Explanation	10%
Total	100%

20. What We Care About Most

The following areas are especially important:

Architecture

Can the candidate design reasonable boundaries between:

- Mobile
- Backend
- Payload
- Database
- RAG
- LLM

Backend

Can they build APIs that are:

- Structured
- Secure
- Validated
- Maintainable

Data Modeling

Do they understand relationships between:

- Users
- Doctors
- Slots
- Appointments
- Unresolved queries

Concurrency

Do they understand why checking availability on the frontend is insufficient?

RAG

Can they build a genuinely grounded RAG system rather than simply sending the user's question directly to an LLM?

AI Reliability

Does the chatbot correctly recognize:

“I don't have enough information.”

instead of confidently inventing an answer?

AI-Assisted Development

Can the candidate use AI as an engineering accelerator rather than blindly accepting generated code?

Explanation

Can the candidate explain what they built and why?

21. Suggested Test Cases

The evaluator should test at least the following.

Appointment

Test 1 — Normal booking

Register → Login → Select doctor → Select slot → Book.

Expected:

Appointment is created successfully.

Test 2 — Same slot

Two users attempt to book the same slot.

Expected:

Exactly one booking succeeds.

Test 3 — Own appointments

User A must not be able to retrieve User B's appointment.

22. RAG Evaluation

Provide the candidate with the agreed healthcare PDFs.

Test at least:

Test 1 — Answerable question

Ask a question whose answer clearly exists in the supplied documents.

Expected:

- Relevant answer
- Answer grounded in retrieved content
- Source shown where practical

Test 2 — Unanswerable question

Ask a question whose answer does not exist in the documents.

Expected:

- No fabricated answer
- Safe fallback
- Question stored in Payload
- Status = New

Test 3 — Weakly related question

Ask something semantically related to the documents but whose specific answer is not actually contained in them.

Expected:

The system should recognize that semantic similarity alone does not necessarily mean the knowledge base contains the answer.

Test 4 — Admin resolution

Admin opens the unresolved query.

Expected:

- Question is visible
 - Admin can enter a human response
 - Status can be changed to Resolved
-

23. Optional Enhancements

These are **not required** and should not negatively affect the candidate if omitted.

Examples:

- Chat history
- Appointment cancellation
- RAG source page references
- Automated tests
- Docker

- Filtering/searching unresolved queries
- Basic logging
- Better appointment scheduling
- Improved UI/UX
- Additional chatbot features

Candidates should prioritize completing the mandatory requirements before implementing optional features.

24. Expected Time

Target implementation time is approximately:

16–20 hours

Candidates should prioritize:

1. Core architecture
2. Authentication
3. Doctor/slot/appointment flow
4. Double-booking protection
5. RAG ingestion
6. RAG retrieval
7. Grounded chatbot answers
8. Unknown-question fallback
9. Unresolved query workflow
10. Basic security and documentation

A partially completed optional feature is preferable to sacrificing a required feature for additional polish.

25. Technical Review

After submission, the candidate may be asked to participate in a **30–45 minute technical walkthrough**.

During the walkthrough, the candidate should be able to:

- Explain the architecture
- Walk through the database models
- Explain the booking flow
- Explain how double booking is prevented
- Explain the RAG ingestion pipeline
- Explain retrieval and relevance logic
- Explain how hallucinations are prevented
- Demonstrate an unanswered question
- Explain the Payload admin workflow
- Discuss AI-generated code
- Identify tradeoffs and possible improvements

The purpose is not to test whether the candidate memorized their code.

It is to determine whether they **understand the system they built**.

26. Final Acceptance Criteria

The submission should satisfy the following baseline:

Appointment

- ☐ User can register
- ☐ User can log in
- ☐ User can view doctors
- ☐ User can view doctor details
- ☐ User can view available slots
- ☐ User can book a slot
- ☐ Same slot cannot be booked twice
- ☐ User can view their appointment
- ☐ Doctors are managed through Payload
- ☐ Slots are managed through Payload

RAG

- ☐ Provided PDFs can be indexed
- ☐ Text is extracted and chunked
- ☐ Embeddings are generated

- ☐ Embeddings/chunks are stored in a vector store
- ☐ User questions perform vector retrieval
- ☐ LLM answers using retrieved context
- ☐ Answerable questions receive grounded answers
- ☐ Source document is shown where practical
- ☐ Unsupported questions are not fabricated
- ☐ Unsupported questions are stored in Payload
- ☐ Unresolved questions have New status
- ☐ Admin can view unresolved questions
- ☐ Admin can enter a human response
- ☐ Admin can mark questions Resolved

Technical

- ☐ Ionic React Android application
 - ☐ Payload CMS
 - ☐ MongoDB
 - ☐ Node.js/TypeScript backend
 - ☐ LLM integration
 - ☐ Vector store
 - ☐ API keys kept on backend
 - ☐ Basic authentication/authorization
 - ☐ User data isolation
 - ☐ README with setup and technical decisions
 - ☐ Candidate can explain the implementation
-

Final Principle

This assignment is intentionally designed to evaluate a **senior full-stack developer working with modern AI-assisted development tools**.

We do not expect the candidate to build a production healthcare platform in 20 hours.

We do expect them to demonstrate that they can take a real-world requirement, design a sensible architecture, use AI tools effectively, build the core system, handle important edge cases, integrate an LLM responsibly, and explain the engineering decisions behind the result.